

Guía para el usuario

Proyecto QuantPostRS

1. Introducción

Esta guía explica cómo preparar y utilizar el repositorio de trabajo formal para el proyecto *QuantPostRS*. Está pensada para usuarios que deseen ejecutar los scripts y agentes localmente, utilizar `codex` para avanzar en las demostraciones y comprender la filosofía de uso que evita el “derrape” (*drift*) en investigaciones colaborativas.

2. Configuración del entorno (*Setup*)

2.1. Requisitos previos

- Sistema operativo Linux con `bash`.
- Instalación de `Cadabra2` y `SageMath` disponibles en la línea de comandos. En muchas distribuciones pueden instalarse con:


```
sudo apt install cadabra2 sagemath
```
- Contar con acceso a `codex-cli`. El binario `codex` debe estar en `$PATH`.
- (*Opcional*) Un compilador `LATEX` para generar PDFs de las revisiones (`pdflatex` o similar).

2.2. Obtención del repositorio

Una vez creado y publicado (ver más abajo), clona el repositorio de trabajo (o copia la carpeta entregada) en una ubicación de tu elección. Por ejemplo:

```
git clone https://github.com/tu-usuario/QuantPostRS_workflow.git
cd QuantPostRS_workflow
```

La estructura inicial debería contener subcarpetas como `corpus/`, `open_questions/`, `claims/`, `scripts/`, `reports/`, `evaluations/`, `interpretations/`, `decisions/`, `working/` y `reviews/`, además de archivos como `AGENTS.md` y `FORMAL_METHODS_GUIDE.md`. Si faltan, créalas manualmente o copia la plantilla suministrada.

2.3. Ejecución de `codex`

Entra en el directorio raíz del proyecto y ejecuta:

```
codex
```

Esto iniciará la interfaz de línea de comandos de Codex. Dentro podrás proporcionar instrucciones siguiendo los contratos y guías del proyecto. Si necesitas dejar la sesión, puedes usar `Ctrl+C`; para retomar, vuelve a ejecutar `codex` y dale contexto sobre dónde quedó la conversación (ejemplo: leer los reportes y scripts existentes antes de continuar).

2.4. Compilación de las revisiones

Si el Agente Revisor genera un fichero `.tex` en `reviews/current/`, puedes compilarlo con:

```
cd reviews/current
pdflatex progress_review.tex
```

o el nombre correspondiente. Esto creará un PDF legible sin alterar el corpus.

3. Filosofía de uso

- **El archivo `.tex` es la autoridad.** No se modifica salvo decisiones explícitas. Toda nueva afirmación debe rastrearse a una pregunta abierta del documento.
- **Corpus inmutable.** Los artefactos en `corpus/` (claims, scripts, reportes, etc.) no se editan ni se eliminan. Para cambiar una idea se crea un nuevo artefacto que indique cómo se relaciona con el anterior.
- **Distinción entre evidencia y trabajo.** Los directorios `working/` y `reviews/` son para borradores y vistas temporales. Los resultados formales y permanentes van a `corpus/`.
- **Control formal antes de la interpretación.** Nunca avances una conclusión física sin haber probado rigurosamente los pasos algebraicos con Cadabra2 y/o SageMath. Usa `codex` como asistente, no como oráculo.
- **Registro de supuestos y contextos.** Documenta siempre qué hipótesis utilizas (signatura, métricas, conmutadores) y conserva los residuos de los cálculos. Esto permite reproducir y auditar las demostraciones.

4. Rol de Git en el workflow

Git cumple un rol técnico-operativo: registra la historia de archivos, permite trabajar en ramas, recuperar estados anteriores, revisar cambios antes de aceptarlos y colaborar con otras personas. No reemplaza al corpus. La distinción conceptual es:

Git registra la historia técnica de los archivos.

El corpus registra la historia conceptual de la investigación.

Por eso, Git y el corpus deben usarse juntos. Git permite saber qué cambió; el corpus debe explicar por qué ese cambio importa.

4.1. Crear el repositorio local desde cero

Si todavía no existe un repositorio, crea una carpeta de trabajo e inicializa Git:

```
mkdir QuantPostRS_workflow
cd QuantPostRS_workflow
git init
```

Luego copia dentro de esta carpeta el documento ‘`.tex`’ de referencia, los archivos de configuración del workflow y las carpetas de trabajo:

```
AGENTS.md
FORMAL_METHODS_GUIDE.md
AGENTS_review_monitor.md
QuantPostRS_DocumentoReferenciaDeContexto_agent_friendly.tex
open_questions/
claims/
scripts/
reports/
evaluations/
interpretations/
decisions/
working/
reviews/
```

Después revisa el estado:

```
git status
```

Si todo está correcto, realiza el primer commit:

```
git add .
git commit -m "Initial QuantPostRS formal workflow setup"
```

Este commit inicial debe representar el punto de partida limpio del proyecto.

4.2. Crear un archivo .gitignore

Conviene crear un archivo ‘.gitignore’ para evitar subir archivos temporales, generados o locales. Una versión inicial razonable es:

```
# LaTeX generated files
*.aux
*.log
*.out
*.toc
*.synctex.gz
*.fls
*.fdb_latexmk

# Python/Sage generated files
__pycache__/
*.pyc
*.sage.py

# Temporary files
*.tmp
*.bak

# Local scratch space
working/scratch/

# Optional generated PDFs
reviews/current/*.pdf
```

Advertencia: no ignores salidas que sean evidencia primaria. Si un archivo ‘.out’ de Cadabra2 o SageMath documenta un resultado importante, debe incorporarse al corpus o quedar resumido dentro de un reporte citable.

Después de crear ‘.gitignore’, registra el cambio:

```
git add .gitignore
git commit -m "Add gitignore for generated and temporary files"
```

4.3. Publicar el repositorio

Para publicar el repositorio en GitHub, GitLab u otro servicio, primero crea un repositorio vacío en la plataforma elegida. No inicialices allí un README si ya lo tienes localmente, para evitar conflictos innecesarios.

Luego conecta tu repositorio local con el remoto. Por ejemplo, si el remoto se llama ‘origin’:

```
git remote add origin git@github.com:TU_USUARIO/QuantPostRS_workflow.git
git branch -M main
git push -u origin main
```

Si usas URL HTTPS en lugar de SSH:

```
git remote add origin https://github.com/TU_USUARIO/QuantPostRS_workflow.git
git branch -M main
git push -u origin main
```

A partir de ese momento, los cambios locales pueden publicarse con:

```
git push
```

Y otras personas pueden obtenerlos con:

```
git clone git@github.com:TU_USUARIO/QuantPostRS_workflow.git
```

4.4. Regla de trabajo antes de usar Codex

Antes de iniciar una sesión con Codex, conviene que el árbol de trabajo esté limpio:

```
git status
```

El estado ideal es:

```
nothing to commit, working tree clean
```

Luego crea una rama para el episodio de trabajo:

```
git checkout -b q002-gamma-trace
```

El nombre de la rama debe indicar el objetivo: ‘q002-gamma-trace’, ‘review-monitor’, ‘cadabra-rules-refactor’, ‘q003-principal-symbol’, etc.

4.5. Usar ramas para sesiones de Codex

Codex debe trabajar preferentemente en una rama específica. Esto permite experimentar sin contaminar 'main'.

Ejemplo:

```
git checkout -b q003-principal-symbol  
codex
```

Al terminar la sesión, revisa qué cambió:

```
git status  
git diff
```

Si el resultado es correcto:

```
git add .  
git commit -m "Add Q003 principal-symbol exploratory claims"  
git checkout main  
git merge q003-principal-symbol  
git push
```

Si Codex modificó algo que no debía, puedes revertir un archivo:

```
git restore ruta/del/archivo
```

Si el experimento completo fue inútil o dañino y no hay nada que preservar:

```
git reset --hard
```

Usa este último comando con cuidado: elimina cambios no comiteados.

4.6. Commits como episodios conceptuales

Los commits deben representar episodios coherentes de investigación, no simples acumulaciones de archivos. Buenos ejemplos:

```
Add baseline gauge invariance audit  
Add corpus mode templates  
Add review monitor agent  
Add Q001 free-limit open question  
Add CLAIM-0010 gamma-trace formalization  
Add EVAL-0010 classifying gamma-trace as partial
```

Evita mensajes ambiguos como:

```
updates  
fix stuff  
more files  
codex changes
```

Un buen commit debe permitir reconstruir qué avance conceptual ocurrió.

4.7. Git y corpus immutable

Git no reemplaza la regla de corpus immutable. Aunque Git permite modificar un archivo y ver su historia, dentro del workflow los artefactos del corpus no deben editarse en sitio. Si un claim, reporte o evaluación queda obsoleto, se crea otro artefacto que lo enlaza formalmente:

```
CLAIM-0010_gamma_trace_EOM.md
EVAL-0010_gamma_trace_partial.md
CLAIM-0017_weaker_gamma_trace_EOM.md
EVAL-0017_gamma_trace_pass.md
DECISION-0008_use_CLAIM-0017_not_CLAIM-0010.md
```

Git registra que aparecieron nuevos archivos. El corpus registra que ‘CLAIM-0017’ corrige, reemplaza o refina a ‘CLAIM-0010’.

4.8. Regla de checkpoint

Una regla operativa recomendable es:

```
Antes de cada sesión Codex, el árbol Git debe estar limpio.
Cada sesión Codex debe ocurrir en una rama nombrada.
Después de la sesión, una persona revisa git diff antes de commitear.
Ningún artefacto immutable del corpus debe modificarse in-place.
Cada commit representa un episodio de investigación coherente.
```

En resumen: Git cuida la integridad técnica del repositorio; el corpus cuida la memoria conceptual del proyecto. Uno protege la cocina; el otro, la receta.

5. Guía de interacción con codex

Para trabajar con codex de manera eficaz, sigue estos principios:

5.1. Lee y comprende el contexto

Antes de solicitar acciones al agente, pídele que lea los archivos clave y reconstruya el estado. Por ejemplo:

```
Leé AGENTS.md y FORMAL_METHODS_GUIDE.md. No modifiques nada.
Leé corpus/index.md y los reportes en corpus/reports/.
Indica qué claims existen y sus estados.
```

Esto asegura que el modelo respete las reglas y no reintroduzca problemas ya resueltos.

5.2. Proponer un claim

Cuando detectes una cuestión abierta, formula un claim mínimo:

```
Identificá la pregunta Q-0002 sobre la estructura de restricciones.
Creá CLAIM-0010 que afirme que la traza gamma de las EOM cargadas se anula idénticamente.
No ejecutes todavía los tests.
```

El claim se almacenará en claims/ con sus metadatos YAML.

5.3. Formalizar y ejecutar

Una vez creado el claim, formalízalo:

Formaliza CLAIM-0010:

- Escribí el script Cadabra2 en `scripts/cadabra/SCRIPT-CADABRA-0010.cdb`.
- Escribí el script SageMath en `scripts/sage/SCRIPT-SAGE-0010.sage`.
- No modifiques scripts compartidos.

Luego ejecutá ambos scripts y guarda los residuos en `reports/REPORT-0010.md`.

Los reportes deben incluir la salida de Cadabra y Sage y cualquier observación.

5.4. Evaluar y clasificar

Después de ejecutar, pide una evaluación:

Evaluá CLAIM-0010 usando REPORT-0010.

Decidí si es TESTED_PASS, TESTED_PARTIAL o TESTED_FAIL.

Registra la evaluación en `evaluations/EVAL-0010.md`.

No uses el claim en la física hasta que la evaluación indique TESTED_PASS.

5.5. Interpretar físicamente

Solo después de un PASS:

Creá una interpretación física INTERP-0007 basada en CLAIM-0010.

Describe cómo la traza gamma impone la eliminación del sector spin 1/2.

5.6. Pedir una revisión

Para obtener un resumen general:

Actuá como Review Monitor.

Modo: Informe de estado de claims.

Generá `reviews/current/claim_status_report.tex` y su manifest.

O bien para una revisión extensa:

Actuá como Review Monitor.

Modo: Revisión completa.

Generá `reviews/current/full_progress_review.tex` con todas las secciones.

6. Ejemplo de workflow

A continuación se presenta un posible flujo de trabajo en forma de prompts para codex:

1. Preparación:

Leé `AGENTS.md`, `FORMAL_METHODS_GUIDE.md`,
`corpus/index.md` y `QuantPostRS_DocumentoReferenciaDeContexto_agent_friendly.tex`.
No modifiques nada. Presenta un resumen de los claims existentes y
las preguntas abiertas.

2. Generar un nuevo claim:

Detectá que la Sección 3.2 del .tex plantea la duda sobre propagación causal en campos magnéticos constantes.
Proponé el claim CLAIM-0015:
"El símbolo principal del operador cargado tiene rango completo para $F_{12} \neq 0$ en el régimen perturbativo."
Crea el archivo de claim con sus metadatos.

3. Formalizar y ejecutar:

Formalizá CLAIM-0015 con scripts Cadabra2 y Sage.
Asegurate de heredar las reglas de gamma y signatura.
Añade pruebas de mutación para detectar errores de signo.
Guarda los resultados en reports/REPORT-0015.md.

4. Evaluar:

Evalúa CLAIM-0015 con base en REPORT-0015.
Decidí su estado y registra la evaluación en evaluations/EVAL-0015.md.

5. Interpretar si procede:

Si CLAIM-0015 tiene estatus TESTED_PASS, crea una interpretación que discuta la implicancia para la propagación en campos magnéticos.

6. Monitor de progreso:

Actuá como Review Monitor.
Modo: Informe de estado de claims.
Generá reviews/current/claim_status_report.tex.

7. Conclusión

Trabajar con codex en este proyecto implica disciplina y trazabilidad. El valor añadido surge de preservar cada idea, prueba y fallo como un artefacto citable, y de insistir en que la matemática respalde a la física en cada paso. Usa esta guía como referencia y adapta el flujo de trabajo a tus necesidades, respetando siempre la filosofía del corpus inmutable y la auditoría formal.